



UNIT-2

CSS3 Basics – Selectors - Box Model -
Backgrounds and Borders -Text Effects
– Advanced Features.

CSS

What is CSS?

- CSS is the language we use to style a Web page.
- CSS stands for Cascading Style Sheets
- CSS describes how HTML elements are to be displayed on screen, paper, or in other media
- CSS saves a lot of work. It can control the layout of multiple web pages all at once
- External stylesheets are stored in CSS files

CSS

- HTML was NEVER intended to contain tags for formatting a web page.
- HTML was created to describe the content of a web page, like:
 <h1>This is a heading</h1>
 <p>This is a paragraph.</p>
- When tags like , and color attributes were added to the HTML 3.2 specification, it became hectic for web developers.
- Development of large websites, where fonts and color information were added to every single page, became a long and expensive process.
- To solve this problem, the World Wide Web Consortium (W3C) created CSS.
- CSS removed the style formatting from the HTML page.

CSS Syntax

- A CSS rule consists of a selector and a declaration block.

```
p {  
  color: red;  
  text-align: center;  
}
```

- ✓ **p** is a selector in CSS (it points to the HTML element you want to style: <p>).
- ✓ **color** is a property, and **red** is the property value
- ✓ **text-align** is a property, and **center** is the property value
- ✓ **{color: red; text-align: center;}** is the declaration

CSS Syntax

- The **selector** points to the HTML element you want to style.
- The **declaration** block contains one or more declarations separated by semicolons.
- Each declaration includes a **CSS property** name and a value, separated by a colon.
- Multiple CSS declarations are separated with **semicolons**, and declaration blocks are surrounded by **curly braces**.

CSS Syntax

- **Example:**

```
<!DOCTYPE html>
<html>
<head>
<style>
p {
  color: red;
  text-align: center;
}
</style>
</head>
<body>

<p>Hello World!</p>
<p>These paragraphs are styled with CSS.</p>

</body>
</html>
```

Output:

Hello World!

These paragraphs are styled with CSS.

- Here, all <p> elements on the page will be center-aligned, with a red text color

How to add CSS

- When a browser reads a style sheet, it will format the HTML document according to the information in the style sheet.
- **Three Ways to Insert CSS:** There are three ways of inserting a style sheet:
 - External CSS
 - Internal CSS
 - Inline CSS

How to add CSS

External CSS:

- With an external style sheet, you can change the look of an entire website by changing just one file!
- Each HTML page must include a reference to the external style sheet file inside the <link> element, inside the head section.
- **Example:**

```
<!DOCTYPE html>
<html>
<head>
<link rel="stylesheet" href="mystyle.css">
</head>
<body>

<h1>This is a heading</h1>
<p>This is a paragraph.</p>

</body>
</html>
```

Output:

This is a heading

This is a paragraph.

How to add CSS

External CSS:

- An external style sheet can be written in any text editor, and must be saved with a .css extension.
- The external .css file should not contain any HTML tags.
- Here is how the "mystyle.css" file looks:
- **Example:**

```
body {  
    background-color: lightblue;  
}  
  
h1 {  
    color: navy;  
    margin-left: 20px;  
}
```

How to add CSS

Internal CSS:

- An internal style sheet may be used if one single HTML page has a unique style.
- The internal style is defined inside the <style> element, inside the head section.

- **Example:**

```
<!DOCTYPE html>
<html>
<head>
<style>
body {
  background-color: linen;
}

h1 {
  color: maroon;
  margin-left: 40px;
}
</style>
</head>
<body>

<h1>This is a heading</h1>
<p>This is a paragraph.</p>

</body>
</html>
```

Output:

This is a heading

This is a paragraph.

How to add CSS

Inline CSS:

- An inline style may be used to apply a unique style for a single element.
- To use inline styles, add the style attribute to the relevant element. The style attribute can contain any CSS property.

- **Example:**

```
<!DOCTYPE html>
<html>
<body>

<h1 style="color:blue;text-align:center;">This is a heading</h1>
<p style="color:red;">This is a paragraph.</p>

</body>
</html>
```

Output:

This is a heading

This is a paragraph.

CSS Selector

- A CSS selector selects the HTML element(s) you want to style.
- CSS selectors are used to "find" (or select) the HTML elements you want to style.
- We can divide CSS selectors into five categories:
 - **Simple selectors** (select elements based on name, id, class)
 - **Combinator selectors** (select elements based on a specific relationship between them)
 - **Pseudo-class selectors** (select elements based on a certain state)
 - **Pseudo-elements selectors** (select and style a part of an element)
 - **Attribute selectors** (select elements based on an attribute or attribute value)

CSS Selector

Basic/ Simple Selectors

• Basic selectors in [CSS](#) are simple tools used to target specific HTML elements for styling. These include selecting by element name (e.g., h1), class (.class Name), ID (#idName), or universally (* for all elements).

1. Universal Selector (*): Selects all elements on the page and applies the same style universally. For example, setting the font color for every element.

Example:

```
<html>
<head>
  <style>
    * {
      color: red;
    }
  </style>
</head>
<body>
  <h1>Header Text</h1>
  <p>Paragraph Text</p>
</body>
</html>
```

Output:

Header Text

Paragraph Text

CSS Selector

2. Element Selector: Targets all elements of a specific type, such as paragraphs or headers. For example, setting a common font size for all paragraphs

- **Example:**

```
<html>
<head>
  <style>
    p {
      font-size: 18px;
    }
  </style>
</head>
<body>
  <p>This paragraph text will be styled with font size 18px.</p>
</body>
</html>
```

Output:

This paragraph text will be styled with font size 18px.

CSS Selector

3. Class Selector: Applies styles to elements with a specific class attribute. For instance, making all buttons have a blue background.

- **Example:**

```
<html>
<head>
  <style>
    .button {
      background-color: blue;
      color: white;
    }
  </style>
</head>
<body>
  <button class="button">Click Me!</button>
</body>
</html>
```

Output:

Click Me!

CSS Selector

4. ID selector: Styles a single element identified by its unique id. For example, changing the background color of a header.

- **Example:**

```
<html>
<head>
  <style>
    #header {
      background-color: aqua;
    }
  </style>
</head>
<body>
  <div id="header">This is the header section.</div>
</body>
</html>
```

Output:

This is the header section.

CSS Selector

Grouping Selector

- The grouping selector selects all the HTML elements with the same style definitions. It will be better to group the selectors, to minimize the code.
- To group selectors, separate each selector with a comma
- Look at the following CSS code (the h1, h2, and p elements have the same style definitions):

- **Example:**

```
<!DOCTYPE html>
<html>
<head>
<style>
h1, h2, p {
  text-align: center;
  color: red;
}
</style>
</head>
<body>

<h1>Hello World!</h1>
<h2>Smaller heading!</h2>
<p>This is a paragraph.</p>

</body>
</html>
```

Output:

Hello World!

Smaller heading!

This is a paragraph.

CSS Combinator

- A combinator is something that explains the relationship between the selectors.
- A CSS selector can contain more than one simple selector. Between the simple selectors, we can include a combinator.
- There are four different combinators in CSS:
 - Descendant combinator (space)
 - Child combinator (>)
 - Next sibling combinator (+)
 - Subsequent-sibling combinator (~)

CSS Combinator

Descendant combinator (space):

- The descendant combinator matches all elements that are descendants of a specified element.
- The following example selects all <p> elements inside <div> elements:

- **Example:**

```
<!DOCTYPE html>
<html>
<head>
<style>
div p {
  background-color: yellow;
}
</style>
</head>
<body>

<h2>Descendant Selector</h2>

<p>The descendant selector matches all elements that are descendants of a specified
element.</p>

<div>
  <p>Paragraph 1 in the div.</p>
  <p>Paragraph 2 in the div.</p>
  <section><p>Paragraph 3 in the div.</p></section>
</div>

<p>Paragraph 4. Not in a div.</p>
<p>Paragraph 5. Not in a div.</p>

</body>
</html>
```

CSS Combinator

Output:

Descendant Selector

The descendant selector matches all elements that are descendants of a specified element.

Paragraph 1 in the div.

Paragraph 2 in the div.

Paragraph 3 in the div.

Paragraph 4. Not in a div.

Paragraph 5. Not in a div.

CSS Combinator

Child combinator (>)

- The child combinator selects all elements that are the children of a specified element.
- The following example selects all <p> elements that are children of a <div> element:

- **Example:**

```
<!DOCTYPE html>
<html>
<head>
<style>
div > p {
  background-color: yellow;
}
</style>
</head>
<body>

<h2>Child Selector</h2>

<p>The child selector (>) selects all elements that are the children of a specified
element.</p>

<div>
  <p>Paragraph 1 in the div.</p>
  <p>Paragraph 2 in the div.</p>
  <section>
    <p>Paragraph 3 in the div (inside a section element).</p>
  </section>
  <p>Paragraph 4 in the div.</p>
</div>

<p>Paragraph 5. Not in a div.</p>
<p>Paragraph 6. Not in a div.</p>

</body>
</html>
```

CSS Combinator

Output:

Child Selector

The child selector ($>$) selects all elements that are the children of a specified element.

Paragraph 1 in the div.

Paragraph 2 in the div.

Paragraph 3 in the div (inside a section element).

Paragraph 4 in the div.

Paragraph 5. Not in a div.

Paragraph 6. Not in a div.

CSS Combinator

Next sibling combinator (+)

- The next sibling combinator is used to select an element that is directly after another specific element.
- Sibling elements must have the same parent element, and "adjacent" means "immediately following". The following example selects the first <p> element that are placed immediately after <div> elements:

- **Example:**

```
<!DOCTYPE html>
<html>
<head>
<style>
div + p {
  background-color: yellow;
}
</style>
</head>
<body>

<h2>Adjacent Sibling Selector</h2>

<p>The + selector is used to select an element that is directly after another specific element.</p>
<p>The following example selects the first p element that are placed immediately after div elements:
</p>

<div>
  <p>Paragraph 2 in the div.</p>
</div>

<p>Paragraph 3. After a div.</p>
<p>Paragraph 4. After a div.</p>

<div>
  <p>Paragraph 6 in the div.</p>
</div>

<p>Paragraph 7. After a div.</p>
<p>Paragraph 8. After a div.</p>
</body>
</html>
```

CSS Combinator

Output:

Adjacent Sibling Selector

The + selector is used to select an element that is directly after another specific element.

The following example selects the first p element that are placed immediately after div elements:

Paragraph 2 in the div.

Paragraph 3. After a div.

Paragraph 4. After a div.

Paragraph 6 in the div.

Paragraph 7. After a div.

Paragraph 8. After a div.

CSS Combinator

Subsequent sibling combinator (~)

- The subsequent-sibling combinator selects all elements that are next siblings of a specified element.
- The following example selects all <p> elements that are next siblings of <div> elements:

- **Example:**

```
<!DOCTYPE html>
<html>
<head>
<style>
div ~ p {
  background-color: yellow;
}
</style>
</head>
<body>
```

```
<h2>General Sibling Selector</h2>
```

```
<p>The general sibling selector (~) selects all elements that are next siblings of a
specified element.</p>
```

```
<p>Paragraph 1.</p>
```

```
<div>
  <p>Paragraph 2.</p>
</div>
```

```
<p>Paragraph 3.</p>
<code>Some code.</code>
<p>Paragraph 4.</p>
```

```
</body>
</html>
```

CSS Combinator

Output:

General Sibling Selector

The general sibling selector (~) selects all elements that are next siblings of a specified element.

Paragraph 1.

Paragraph 2.

Paragraph 3.

Some code.

Paragraph 4.

CSS Combinator

Name	Sign	Example	Example Result
Child combinator	>	div > p	Selects every <p> element that are direct children of a <div> element
Descendant combinator	(single space)	div p	Selects all <p> elements inside <div> elements
Namespace separator		ns h2	Selects all <h2> elements in namespace ns
Next-sibling combinator	+	div + p	Selects the first <p> element that is placed immediately after <div> elements
Selector list	,	div, p	Selects all <div> elements and all <p> elements
Subsequent-sibling combinator	~	p ~ ul	Selects all elements that are preceded by a <p> element

Pseudo-class

CSS Pseudo-classes:

- A pseudo-class is used to define a special state of an element.
- For example, it can be used to:
 - ✓ Style an element when a user moves the mouse over it
 - ✓ Style visited and unvisited links differently
 - ✓ Style an element when it gets focus
 - ✓ Style valid/invalid/required/optional form elements

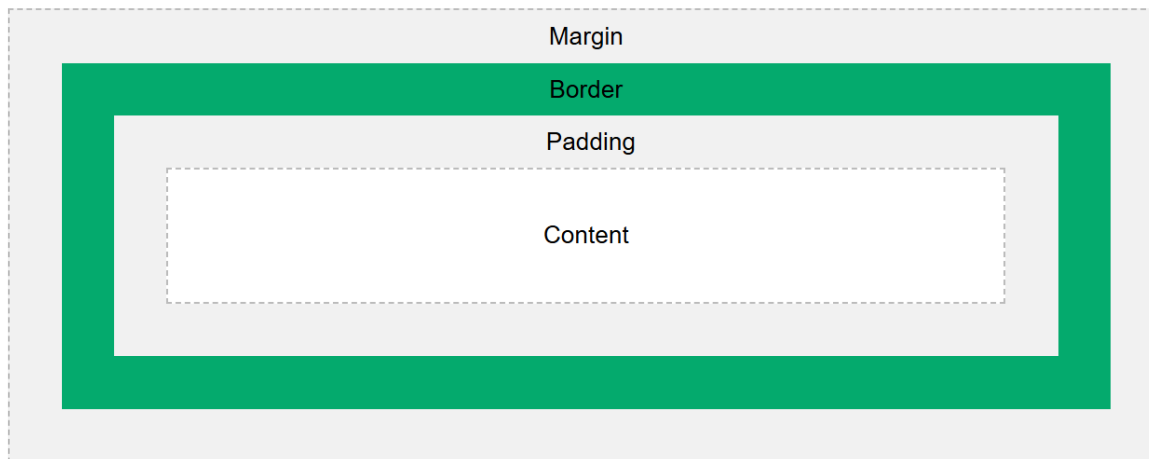
Pseudo-element

CSS Pseudo-element:

- A CSS pseudo-element is used to style specific parts of an element.
- For example, it can be used to:
 - ✓ Style the first letter or line, of an element
 - ✓ Insert content before or after an element
 - ✓ Style the markers of list items
 - ✓ Style the viewbox behind a dialog box

Box Model

- In CSS, the term "box model" is used for about design and layout.
- The CSS box model is essentially a box that wraps around every HTML element. It consists of: content, padding, borders and margins. The image below illustrates the box model:



Box Model

Explanation of the different parts:

- **Content** - The content of the box, where text and images appear
- **Padding** - Clears an area around the content. The padding is transparent
- **Border** - A border that goes around the padding and content
- **Margin** - Clears an area outside the border. The margin is transparent
- The box model allows us to add a border around elements, and to define space between elements.

Box Model

1. Content Area

- The content area is the central part of the CSS box model, containing the main content (e.g., text, images, videos, or elements like <p> or).
- It can be styled with CSS properties like height and width.
- The content edge refers to the four edges of the content area
 - ✓ Left content edge
 - ✓ Right content edge
 - ✓ Top content edge
 - ✓ Bottom content edge

Box Model

2. Padding Area

- The padding area is the space between the content and the border of an element.
- The distance between the [content edge](#) and the border is the padding.
- The border marks the end of the padding area.
- The padding area contributes to the element's total dimensions.
- It Padding can be adjusted using [CSS properties](#).
- It works similarly with box-sizing: content-box and box-sizing: border-box, but with slight calculation differences.

Box Model

3. Border Area

- The area that marks the end of an element is called as the border it is the outer fencing for the element.
- The default border properties are provided in CSS to control the thickness of this outer fencing.
- The border area also add 's up to the complete height and width of the element.
- The more the border width the more will be the height or width of the element..

4. Margin Area

- The area outside the border of an element is called the margin area.
- Basically this area depends on the parent of the element.
- The distance between the border of the parent element and the border of the child element is called as the margin.
- CSS has provides certain margin properties to get control over this scenario.

Box Model

Example

```
<!DOCTYPE html>
<html>
<head>
<style>
div {
  background-color: lightgrey;
  width: 300px;
  border: 15px solid green;
  padding: 50px;
  margin: 20px;
}
</style>
</head>
<body>

<h2>Demonstrating the Box Model</h2>

<p>The CSS box model is essentially a box that wraps around every HTML element. It
consists of: borders, padding, margins, and the actual content.</p>

<div>This text is the content of the box. We have added a 50px padding, 20px margin and
a 15px green border. In CSS, the term "box model" is used for about design and layout.
The CSS box model is essentially a box that wraps around every HTML element. It consists
of: content, padding, borders and margins. </div>

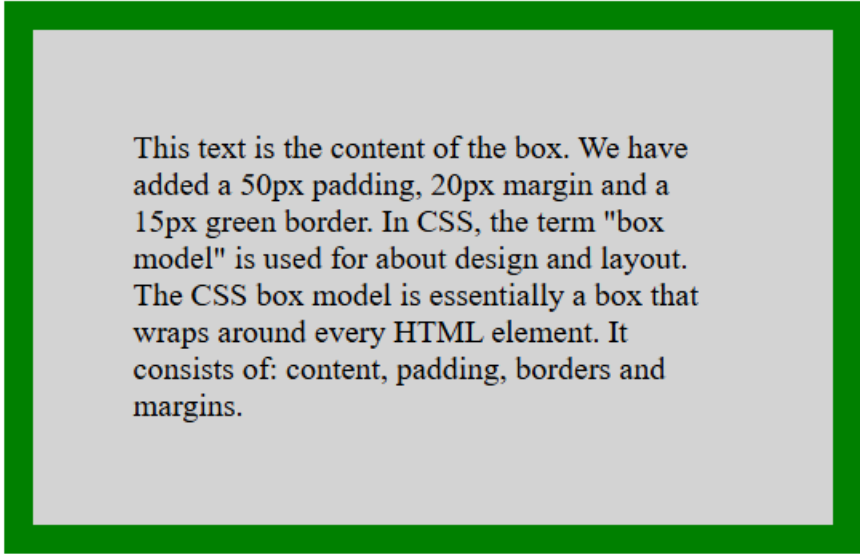
</body>
</html>
```

Box Model

Output:

Demonstrating the Box Model

The CSS box model is essentially a box that wraps around every HTML element. It consists of: borders, padding, margins, and the actual content.



This text is the content of the box. We have added a 50px padding, 20px margin and a 15px green border. In CSS, the term "box model" is used for about design and layout. The CSS box model is essentially a box that wraps around every HTML element. It consists of: content, padding, borders and margins.

Box Model

Width and Height of an Element

- When you set the width and height properties of an element with CSS, you just set the width and height of the **content area**. To calculate the total width and height of an element, you must also include the padding and borders.
- Example
- This <div> element will have a total width of 350px and a total height of 80px:
- ```
div {
 width: 320px;
 height: 50px;
 padding: 10px;
 border: 5px solid gray;
 margin: 0;
}
```
- Here is the calculation:

# Box Model

## Width and Height of an Element

- When you set the width and height properties of an element with CSS, you just set the width and height of the **content area**. To calculate the total width and height of an element, you must also include the padding and borders.
- *Example*

This <div> element will have a total width of 350px and a total height of 80px:

```
div {
 width: 320px;
 height: 50px;
 padding: 10px;
 border: 5px solid gray;
 margin: 0;
}
```

### *Calculation:*

320px (width of content area)  
+ 20px (left padding + right padding)  
+ 10px (left border + right border)  
**= 350px (total width)**

50px (height of content area)  
+ 20px (top padding + bottom padding)  
+ 10px (top border + bottom border)  
**= 80px (total height)**

# Box Model

## Width and Height of an Element

The total width of an element should be calculated like this:

- $\text{Total element width} = \text{width} + \text{left padding} + \text{right padding} + \text{left border} + \text{right border}$

The total height of an element should be calculated like this:

- $\text{Total element height} = \text{height} + \text{top padding} + \text{bottom padding} + \text{top border} + \text{bottom border}$

**Note:** The margin property also affects the total space that the box will take up on the page, but the margin is not included in the actual size of the box. The box's total width and height stops at the border.

# Box Model

## Box Sizing Property in CSS

There are two type's of box-sizing properties in CSS

### 1. Content-Box(default property)

- When the user set's the value of the [box-sizing property](#) for an element as [content-box](#) or even if user do not set's it ,it remains by default as content-box and in the actual height and width of the element the dimensions of the content area as well as the padding area is added to constitute the final dimensions of the element.

### 2. Border-Box

- When the box-sizing property is set as [border-box](#) the actual dimensions of the element's remains same as that of the actual dimensions set by the user. The difference it makes is just that the size of the content area get's altered in a manner so that it could accommodate the [padding area](#) and the [border area](#) so the resultant could be equal to the actual dimensions entered by the user.

# CSS Border

- The CSS border properties allow you to specify the style, width, and color of an element's border.

## CSS Border Style:

The **border-style** property specifies what kind of border to display.

The following values are allowed:

- **dotted** - Defines a dotted border
- **dashed** - Defines a dashed border
- **solid** - Defines a solid border
- **double** - Defines a double border
- **groove** - Defines a 3D grooved border. The effect depends on the border-color value
- **ridge** - Defines a 3D ridged border. The effect depends on the border-color value
- **inset** - Defines a 3D inset border. The effect depends on the border-color value
- **outset** - Defines a 3D outset border. The effect depends on the border-color value
- **none** - Defines no border
- **hidden** - Defines a hidden border

The **border-style** property can have from one to four values (for the top border, right border, bottom border, and the left border).

## Example:

```
<!DOCTYPE html>
<html>
<head>
<style>
p.one {
 border-style: solid; border-width: 5px;
}
p.two {
 border-style: solid; border-width: medium;
}
p.three {
 border-style: dotted; border-width: 2px;
}
p.four {
 border-style: dotted; border-width: thick;
}
</style>
</head>
<body>
<h2>The border-width Property</h2>
<p>This property specifies the width of the four borders:</p>

<p class="one">Some text.</p>
<p class="two">Some text.</p>
<p class="three">Some text.</p>
<p class="four">Some text.</p>

<p>Note: The "border-width" property does not work if it is used alone.
Always specify the "border-style" property to set the borders first.</p>
</body>
</html>
```

## *Output:*

### **The border-width Property**

This property specifies the width of the four borders:

Some text.

Some text.

Some text.

Some text.

**Note:** The "border-width" property does not work if it is used alone. Always specify the "border-style" property to set the borders first.

## Border Width:

- The **border-width** property can have from one to four values (for the top border, right border, bottom border, and the left border)

```
<!DOCTYPE html>
<html>
<head>
<style>
p.one {
 border-style: solid;
 border-width: 5px 20px; /* 5px top and bottom, 20px on the sides */
}
p.two {
 border-style: solid;
 border-width: 20px 5px; /* 20px top and bottom, 5px on the sides */
}
</style>
</head>
<body>
```

```
<h2>The border-width Property</h2>
```

```
<p>The border-width property can have from one to four values (for the top border, right border, bottom border, and the left border):</p>
```

```
<p class="one">Some text.</p>
```

```
<p class="two">Some text.</p>
```

```
</body>
```

```
</html>
```

### *Output:*

#### **The border-width Property**

The border-width property can have from one to four values (for the top border, right border, bottom border, and the left border):

Some text.

Some text.

## Border Color:

The **border-color** property is used to set the color of the four borders.

The color can be set by:

- name - specify a color name, like "red"
- HEX - specify a HEX value, like "#ff0000"
- RGB - specify a RGB value, like "rgb(255,0,0)"
- HSL - specify a HSL value, like "hsl(0, 100%, 50%)"
- transparent

**Note:** If **border-color** is not set, it inherits the color of the element.

### Example

```
<!DOCTYPE html>
<html>
<head>
<style>
p.one {
 border-style: solid;
 border-color: red;
}
</style>
</head>
<body>
```

```
<h2>The border-color Property</h2>
```

```
<p>This property specifies the color of the four borders:</p>
```

```
<p class="one">A solid red border</p>
```

```
<p>Note: The "border-color" property does not work if it is used alone. Use the
"border-style" property to set the borders first.</p>
```

```
</body>
```

```
</html>
```

### Output:

#### The border-color Property

This property specifies the color of the four borders:

A solid red border

**Note:** The "border-color" property does not work if it is used alone. Use the "border-style" property to set the borders first.

# Shorthand Border Property

- To shorten the code, it is also possible to specify all the individual border properties in one property.
- The **border** property is a shorthand property for the following individual border properties:
  - border-width
  - border-style (required)
  - border-color

- **Example**

```
<!DOCTYPE html>
<html>
<head>
<style>
p {
 border: 5px solid red;
}
</style>
</head>
<body>

<h2>The border Property</h2>

<p>This property is a shorthand property for border-width, border-style, and border-color.</p>

</body>
</html>
```

## **Output:**

### **The border Property**

This property is a shorthand property for border-width, border-style, and border-color.

# Rounded Borders:

- *Example*

```
<!DOCTYPE html>
<html>
<head>
<style>
p.round1 {
 border: 2px solid red;
 border-radius: 5px;
 padding: 5px;
}
p.round2 {
 border: 2px solid red;
 border-radius: 8px;
 padding: 5px;
}
p.round3 {
 border: 2px solid red;
 border-radius: 12px;
 padding: 5px;
}
</style>
</head>
<body>

<h2>The border-radius Property</h2>
<p>This property is used to add rounded borders to an element:</p>

<p class="round1">Round border</p>
<p class="round2">Rounder border</p>
<p class="round3">Roundest border</p>

</body>
</html>
```

# CSS Rounded Borders

## *Output:*

Normal border

Round border

Rounder border

Roudest border

- The `border-radius` property is used to add rounded borders to an element:

# CSS Background

- The CSS background properties are used to add background effects for elements. With CSS, a color is most often specified by: color name, HEX value, RGB triplet value.

## CSS background-color:

The `background-color` property specifies the background color of an element.

### *Example:*

```
<!DOCTYPE html>
<html>
<head>
<style>
body {
 background-color: lightblue;
}
</style>
</head>
<body>

<h1>Hello World!</h1>

<p>This page has a light blue background color!</p>

</body>
</html>
```

### *Output:*

**Hello World!**

This page has a light blue background color!

# CSS Background

You can set the background color for any HTML elements:

## **Example:**

```
<!DOCTYPE html>
<html>
<head>
<style>
h1 {
 background-color: green;
}

div {
 background-color: lightblue;
}

p {
 background-color: yellow;
}
</style>
</head>
<body>

<h1>CSS background-color example!</h1>
<div>
This is a text inside a div element.
<p>This paragraph has its own background color.</p>
We are still in the div element.
</div>

</body>
</html>
```

## **Output:**

CSS background-color example!

This is a text inside a div element.

This paragraph has its own background color.

We are still in the div element.

# CSS Background

## CSS background-image:

- The **background-image** property specifies an image to use as the background of an element. By default, the image is repeated so it covers the entire element.

### Example

```
<!DOCTYPE html>
<html>
<head>
<style>
body {
 background-image: url("paper.gif");
}
</style>
</head>
<body>

<h1>Hello World!</h1>

<p>This page has an image as the background!</p>

</body>
</html>
```

### Output:

**Hello World!**

This page has an image as the background!

# CSS Background

## CSS background-repeat:

- By default, the `background-image` property repeats an image both horizontally and vertically.
- Some images should be repeated only horizontally or vertically, or they will look strange, like this:

### Example

```
<!DOCTYPE html>
<html>
<head>
<style>
body {
 background-image: url("gradient_bg.png");
}
</style>
</head>
<body>

<h1>Hello World!</h1>
<p>Strange background image...</p>

</body>
</html>
```

### Output:

**Hello World!**

Strange background image...



# CSS Background

## CSS background-repeat:

- If the image above is repeated only horizontally (**background-repeat: repeat-x;**)
- To repeat an image vertically, set **background-repeat: repeat-y;**
- Showing the background image only once is also specified by the **background-repeat** property :

```
body {
 background-image: url("img_tree.png");
 background-repeat: no-repeat;
}
```

- The **background-position** property is used to specify the position of the background image.
- Example: Position the background image in the top-right corner:

```
body {
 background-image: url("img_tree.png");
 background-repeat: no-repeat;
 background-position: right top;
}
```

# CSS Background

## CSS background-Attachment:

- The `background-attachment` property specifies whether the background image should scroll or be fixed (will not scroll with the rest of the page)
- **Example:** Specify that the background image should be fixed:

```
body {
 background-image: url("img_tree.png");
 background-repeat: no-repeat;
 background-position: right top;
 background-attachment: fixed;
}
```

- **Example:** Specify that the background image should scroll with the rest of the page:

```
body {
 background-image: url("img_tree.png");
 background-repeat: no-repeat;
 background-position: right top;
 background-attachment: scroll;
}
```

# CSS Background

## CSS background- shorthand property:

- To shorten the code, it is also possible to specify all the background properties in one single property. This is called a shorthand property.
- Instead of writing:

```
body {
 background-color: #ffffff;
 background-image: url("img_tree.png");
 background-repeat: no-repeat;
 background-position: right top;
}
```

You can use the shorthand property **background**:

### *Example:*

- Use the shorthand property to set the background properties in one declaration:

```
body {
 background: #ffffff url("img_tree.png") no-repeat right top;
}
```

# CSS Text Effects

## CSS Text Overflow:

- The CSS `text-overflow` property specifies how overflowed content that is not displayed should be signaled to the user.

It can be clipped:

This is some long text that

or it can be rendered as an ellipsis (...):

This is some long text t...

# CSS Text Effects

- *Example:*

```
<!DOCTYPE html>
<html>
<head>
<style>
p.test1 {
 white-space: nowrap;
 width: 200px;
 border: 1px solid #000000;
 overflow: hidden;
 text-overflow: clip;
}

p.test2 {
 white-space: nowrap;
 width: 200px;
 border: 1px solid #000000;
 overflow: hidden;
 text-overflow: ellipsis;
}
</style>
```

# CSS Text Effects

```
<body>
```

```
<h1>The text-overflow Property</h1>
```

```
<p>The following two paragraphs contains a long text that will not fit in the box.</p>
```

```
<h2>text-overflow: clip:</h2>
```

```
<p class="test1">This is some long text that will not fit in the box</p>
```

```
<h2>text-overflow: ellipsis:</h2>
```

```
<p class="test2">This is some long text that will not fit in the box</p>
```

```
</body>
```

```
</html>
```

## ***Output:*** The text-overflow Property

The following two paragraphs contains a long text that will not fit in the box.

**text-overflow: clip:**

This is some long text that will

**text-overflow: ellipsis:**

This is some long text that ...

# CSS Text Effects

## CSS Word-wrapping:

- The CSS **word-wrap** property allows long words to be able to be broken and wrap onto the next line.

If a word is too long to fit within an area, it expands outside:

This paragraph  
contains a very long  
word:  
thisisaveryveryveryveryveryverylongword.  
The long word will  
break and wrap to  
the next line.

The word-wrap property allows you to force the text to wrap - even if it means splitting it in the middle of a word:

This paragraph  
contains a very long  
word:  
thisisaveryveryveryv  
eryveryverylongword  
. The long word will  
break and wrap to  
the next line.

# CSS Text Effects

## *Example:*

```
<!DOCTYPE html>
<html>
<head>
<style>
p.test {
 width: 11em;
 border: 1px solid #000000;
 word-wrap: break-word;
}
</style>
</head>
<body>

<h1>The word-wrap Property</h1>

<p class="test">This paragraph contains a very long word:
thisisaveryveryveryveryveryveryverylongword. The long word will break and wrap to the next
line.</p>

</body>
</html>
```

# CSS Text Effects

*Output:*

## The word-wrap Property

This paragraph contains a very long word:  
thisisaveryveryveryveryveryveryverylongword. The long word will break and wrap to the next line.

# CSS Text Effects

## CSS Word-Breaking:

The CSS `word-break` property specifies line breaking rules.

This paragraph contains some text. This line will-break-at-hyphens.

This paragraph contains some text. The lines will break at any character.

# CSS Text Effects

## CSS Word-Breaking:

```
<!DOCTYPE html>
<html>
<head>
<style>
p.test1 {
 width: 140px;
 border: 1px solid #000000;
 word-break: keep-all;
}

p.test2 {
 width: 140px;
 border: 1px solid #000000;
 word-break: break-all;
}
</style>
</head>
<body>

<h1>The word-break Property</h1>

<p class="test1">This paragraph contains some text. This line will-break-at-hyphens.</p>

<p class="test2">This paragraph contains some text. The lines will break at any
character.</p>

</body>
</html>
```

# CSS Text Effects

## CSS Writing mode:

The CSS `writing-mode` property specifies whether lines of text are laid out horizontally or vertically.

Some text with a span element with a `vertical-rl` writing-mode.

# CSS Text Effects

## CSS Writing mode:

```
<!DOCTYPE html>
<html>
<head>
<style>
p.test1 {
 writing-mode: horizontal-tb;
}

span.test2 {
 writing-mode: vertical-rl;
}

p.test2 {
 writing-mode: vertical-rl;
}
</style>
</head>
<body>
<h1>The writing-mode Property</h1>

<p class="test1">Some text with default writing-mode.</p>

<p>Some text with a span element with a vertical-rl writing-
mode.</p>

<p class="test2">Some text with writing-mode: vertical-rl.</p>

</body>
</html>
```

# CSS Text Effects

## CSS Writing mode:

### The writing-mode Property

Some text with default writing-mode.

Some text with a span element with a vertical-rl writing-mode.

Some text with writing-mode: vertical-rl.

# CSS Text Effects

## CSS Text Effect Properties

- The following table lists the CSS text effect properties:

Property	Description
<u><a href="#">text-justify</a></u>	Specifies how justified text should be aligned and spaced
<u><a href="#">text-overflow</a></u>	Specifies how overflowed content that is not displayed should be signaled to the user
<u><a href="#">word-break</a></u>	Specifies line breaking rules for non-CJK scripts
<u><a href="#">word-wrap</a></u>	Allows long words to be able to be broken and wrap onto the next line
<u><a href="#">writing-mode</a></u>	Specifies whether lines of text are laid out horizontally or vertically

# Exercise

## 1. Design a Profile Card

Create a responsive **profile card** with the following specifications:

- Fixed width: 350px
- Background image of your choice (use a placeholder like "profile-bg.jpg")
- Use box-sizing: border-box
- Padding: 20px, Border: 3px solid #666, Border-radius: 15px
- Include a heading and a paragraph inside the card
- Apply box-shadow to add depth

# Exercise

1. Design a card layout using HTML and CSS with the following requirements:
  - The card should be 300px wide with rounded corners (10px).
  - It should have a box-shadow to create a lifted appearance.
  - The background should be a linear gradient from light blue to white.
  - Include a heading inside the card that uses a glowing text-shadow effect.
  - Inside the card, there should be a paragraph with 20px padding.
  - When the user hovers over the card, it should slightly scale up and change the cursor to a pointer.
2. Write CSS using pseudo-classes and pseudo-elements to:
  - Highlight every third list item with a different background
  - Add a decorative icon before each list item
3. Create a div with:
  - Gradient background from blue to purple (left to right)
  - Rounded corners of 12px
  - A shadowed border effect
4. Write CSS to create a dashed border of 3px in orange around all `<textarea>` elements when they are focused.
5. Write CSS to apply a background image to a section element. The image should not repeat, should cover the entire section, and remain fixed during scrolling.
6. Create a div with:
  - Gradient background from blue to purple (left to right)
  - Rounded corners of 12px
  - A shadowed border effect
7. Write CSS to create a dashed border of 3px in orange around all `<textarea>` elements when they are focused.